Université Abdelmalek Essaâdi
Faculté des sciences et techniques de Tanger
جامعة عبد المالك السعدي
كلية العلوم والتقنيات

RAPPORT DE PROJET DE FIN DE MODULE

Filière : Licence IDAI

Application Web de Réservation des Ressources d'un Data Center

---

PROJET DE FIN DE MODULE : GESTION DES RESSOURCES D'UN DATA CENTER

Encadré par :

Prof. Yasyn EL YUSUFI
Prof. M'hamed AIT KBIR

Réalisé par :

ZIOUANI Doha
OUTAHAR Doha
ELHADI Ahlam
GRAOUI Nawras

Année scolaire : 2025 – 2026

Résumé

Le présent rapport s'inscrit dans le cadre de la conception et de la réalisation d'une **application Web de gestion des ressources d'un Data Center**, développée avec Laravel/PHP et une base de données MySQL. L'objectif est de mettre à disposition une plateforme centralisée permettant la consultation des ressources, la création de demandes de réservation sur une période donnée, ainsi que le suivi du cycle de vie des réservations. L'application organise ces fonctionnalités autour de plusieurs profils d'accès, avec un espace public de consultation, un espace utilisateur pour gérer ses réservations, et un espace d'administration dédié à la supervision du parc matériel et au traitement des demandes.

Au-delà de l'implémentation technique, le projet adopte une démarche structurée intégrant la modélisation de la base de données, la séparation des responsabilités selon l'architecture MVC de Laravel, et l'intégration de mécanismes clés tels que la détection automatique des conflits de réservation (chevauchement temporel) via un service dédié, ainsi que la traçabilité des actions grâce à un journal d'activité (Activity Log). L'administration permet notamment la gestion des ressources (ajout, suppression, bascule d'état/maintenance), la validation ou le refus des réservations, et l'affichage d'indicateurs sur un tableau de bord. Enfin, le projet ouvre des perspectives d'évolution, en particulier la mise en place de notifications internes complètes et le renforcement des statistiques pour les rendre entièrement dynamiques et exploitables à grande échelle.

Table des matières

Glossaire **5**

1 Introduction **8**
1.0.1 Contexte général du projet : Application de Réservation des Ressources d'un Data Center 8
1.0.2 Objectif du Projet 8
1.0.3 Problématique 8
1.0.4 Les Outils utilisés 10

2 Méthodologie et rôle individuel **11**
2.1 Approche méthodologique adoptée au sein de l'équipe 11
2.2 Positionnement de notre rôle et responsabilités principales 12

3 Conception et Développement **14**
3.1 Conception et Développement du l'Application Web de Réservation des Ressources du Data Center 14
3.1.1 Objectif et principes de conception 14
3.2 Diagrammes UML / Merise 14
3.2.1 Diagramme de cas d'utilisation (Use Case) 14
3.2.2 Use Case – Gestion des ressources d'un Data Center 16
3.2.3 Diagramme de classes 17
3.3 Modélisation de la base de données 17
Description 17
3.3.1 Structure de Projet 18
3.3.2 Structure du projet (résumé) 19
3.3.3 Fichiers importants 20

4 Cahier des charges **21**
4.1 Contexte et besoin 21
4.1.1 Objectifs 21
4.1.2 Acteurs et rôles 21
Invité 21
Utilisateur /USER (interne) 21
Administrateur /MANAGER (Admin) 22

4.1.3 Fonctionnalités principales 22
Gestion des ressources (catalogue) 23
Système de réservation et gestion des conflits 23
Tableau de bord et statistiques 23
Logs et traçabilité (audit) 23
Notifications(pour un user)/Historique (pour Admin , User ..) 23
4.1.4 Fonctionnalités implémentées 23
4.1.5 Choix technologiques 24
Frontend 24
Backend 24
Base de données 24

5 Réalisation **25**
5.1 Implémentation 25
5.1.1 Architecture du projet 25
5.1.2 Fonctionnalités 25
5.1.3 Règles de gestion importantes 25
Conflits de réservation (overlapping) 25
Gestion des statuts 25
Permissions (RBAC) 26
5.2 Sécurité et contrôle d'accès 26
5.2.1 Middleware d'authentification 26
5.2.2 Rôles et permissions 26
5.2.3 Protection des routes d'administration 26
5.2.4 Validation des formulaires 26
5.2.5 Gestion des sessions (si stockées en base) 26
5.3 Tests et scénarios 27
5.3.1 Scénarios de test 27

6 Les Captures d'écran de L'Application **28**
6.1 DataCenter Reservation 28

7 Conclusion générale **33**
7.1 Conclusion 33

Liste des figures

3.1 Diagramme de cas d'utilisation du système de réservation des ressources d'un Data Center (rôles : Invité, Utilisateur, Manager, Administrateur).. . . . 16
3.2 Diagramme de classes du système de réservation des ressources d'un Data Center, présentant les entités principales (*User, Resource, Reservation, ActivityLog*), leurs attributs, méthodes et relations (réservations, journalisation et cible polymorphe) 17
3.3 MCD (Merise) du système de réservation des ressources du Data Center . . 18

4.1 Les roles 22

6.1 Interface d'authentification (login) 28
6.3 Interface d'un compte cree 29
6.2 Interface pour Cree un Compte 29
6.4 Interface d'historique des actions realiser dans l'application 30
6.5 Interface de Creation d'une reservation 30
6.6 Interface de Management des ressources 30
6.7 Interface de Management des reservations 30
6.8 Interface de suivis d'une reservation 31
6.9 Interface de Confirmation des details d'une Reservation dans le Data Center 31
6.10 Interface des details Techniques d'un resource 31
6.11 Interface d'administration pour la gestion des reservations 32
6.12 Interface apres log out (switch automatique au mode invite) 32

Glossaire

Backend
Partie serveur d'une application, chargée du traitement des données, de la logique métier et de la communication avec la base de données.

Application Web
Application accessible via un navigateur, permettant la consultation des ressources, la gestion des réservations et l'administration sans installation locale.

Authentification (Authentication)
Mécanisme vérifiant l'identité de l'utilisateur (connexion) afin d'accéder aux fonctionnalités protégées.

Autorisation (Authorization / RBAC)
Contrôle des actions possibles selon le rôle de l'utilisateur (admin/manager/user), afin de protéger les fonctionnalités sensibles.

Blade (Template Engine)
Moteur de templates de Laravel utilisé pour générer les pages HTML (`.blade.php`) côté serveur.

CRUD
Ensemble des opérations de base sur les données : *Create* (créer), *Read* (lire), *Update* (modifier), *Delete* (supprimer).

Data Center
Infrastructure regroupant des ressources informatiques (serveurs, VM, stockage, réseau) destinées à fournir des services et des environnements de travail.

Dashboard (Tableau de bord)
Interface de synthèse affichant des indicateurs (nombre de ressources, réservations, états, ac-

tivités récentes) pour le suivi global.

Détection de conflit (Overlapping)
Vérification empêchant deux réservations simultanées d'une même ressource lorsque leurs périodes se chevauchent.

Disponibilité (Availability)
Capacité d'une ressource à être réservée sur une période donnée, dépendant de son état et de l'absence de conflit.

Eloquent ORM
ORM de Laravel permettant de manipuler les tables via des modèles (ex. `User`, `Resource`) sans écrire du SQL brut.

ENUM
Type de champ limitant les valeurs possibles (ex. rôles, statuts, types), afin d'éviter des valeurs incohérentes.

État d'une ressource (Resource Status)
Valeur indiquant la situation de la ressource : `available` (disponible), `reserved` (réservée), `maintenance` (maintenance), `down` (hors service).

Laravel
Framework PHP structurant l'application selon l'architecture MVC et fournissant routage, sécurité, ORM, validation, sessions et outils de développement.

Middleware
Composant exécuté avant l'accès à une route (ex. vérifier qu'un utilisateur est authentifié ou possède le rôle admin/manager).

Migration
Fichier décrivant la création et l'évolution des tables (colonnes, index, contraintes), garantissant une base versionnée et reproductible.

MySQL
Système de gestion de base de données relationnelle stockant les utilisateurs, ressources, réservations et journaux d'activité.

Réservation
Demande d'utilisation d'une ressource sur une période donnée, suivie par un statut (`pending`, `approved`, `rejected`, `active`, `finished`, `cancelled`).

Ressource
Équipement ou service réservable (serveur, VM, stockage, réseau) identifié par un type, un état et des caractéristiques techniques.

Rôle (Role)
Attribut déterminant le niveau d'accès d'un utilisateur : `user`, `manager`, `admin`.

Route
Définition d'une URL et de son traitement (controller/méthode), éventuellement protégée par middleware.

Validation Laravel (FormRequest)
Mécanisme de validation côté serveur contrôlant les champs envoyés par les formulaires (dates, formats, champs obligatoires).

Chapitre 1

Introduction

1.0.1 Contexte général du projet : Application de Réservation des Ressources d'un Data Center

Le projet *Application Web de Réservation des Ressources d'un Data Center* vise la gestion d'un Data Center moderne implique la manipulation d'un parc matériel complexe et coûteux. Actuellement, de nombreuses organisations font face à des difficultés pour suivre la disponibilité de leurs ressources en temps réel, ce qui peut entraîner des conflits d'usage ou une sous-utilisation des équipements. C'est dans ce contexte que s'inscrit notre mini-projet : répondre à un besoin critique de visibilité et d'organisation. Le projet simule un environnement réel où des ingénieurs, des enseignants ou des doctorants doivent solliciter des ressources spécifiques pour leurs travaux, sous la supervision d'une équipe technique et administrative.

1.0.2 Objectif du Projet

L'objectif majeuru projet d est de mettre en place une solution logicielle robuste basée sur le framework Laravel. Plus spécifiquement, il s'agit de : Concevoir une base de données relationnelle capable de stocker l'historique complet des réservations. Développer une interface web "responsive" et ergonomique en utilisant exclusivement du CSS et JavaScript personnalisés (sans frameworks externes). Implémenter un système de notifications et de statistiques pour permettre aux administrateurs de piloter efficacement le Data Center..

1.0.3 Problématique

La problématique principale est la suivante : comment concevoir une application Web centralisée permettant de gérer la réservation et l'allocation des ressources d'un Data Center (serveurs, VM, stockage, réseau) en garantissant, en temps réel, la disponibilité des ressources, la prévention des chevauchements de réservations (overlapping) et le respect de droits d'accès différenciés selon les rôles (utilisateur, responsable, administrateur) ? Pour répondre à cette problématique, le projet met en place une base de données relationnelle structurée, un cycle de vie clair des réservations (en attente, approuvée, refusée, active, termi-

née, annulée), un mécanisme de vérification automatique des conflits temporels, ainsi qu'un contrôle d'accès via authentification et middleware, complété par un système de traçabilité (journal d'activité) pour suivre les actions sensibles.

Un défi particulier réside dans la gestion des situations de concurrence et d'exploitation : plusieurs utilisateurs peuvent solliciter simultanément la même ressource sur des périodes proches, tandis que certaines ressources peuvent devenir indisponibles (maintenance, panne, désactivation). Il est donc impératif de concevoir une stratégie qui assure la cohérence des décisions (validation/refus), limite les incohérences liées aux mises à jour concurrentes, et conserve un historique fiable des changements (justifications, notes d'administration, logs), tout en offrant une interface claire permettant aux utilisateurs de suivre leurs demandes et à l'administration de superviser efficacement l'occupation globale du Data Center.

1.0.4 Les Outils utilisés

Apache est un serveur web qui fait le pont entre l'utilisateur et le site. Il reçoit les demandes du navigateur et renvoie les pages de l'application. C'est le moteur qui permet au site d'être accessible sur un réseau.

Laravel est le framework utilisé pour coder l'application. C'est une boîte à outils qui facilite le développement en proposant des fonctions prêtes à l'emploi pour la sécurité, les bases de données et la gestion des utilisateurs.

Nginx autre serveur web, concurrent d'Apache. Il est connu pour être très rapide et capable de gérer énormément de visiteurs en même temps tout en consommant très peu de mémoire.

Visual Studio Code est un éditeur de code extensible (Windows, Linux, macOS). Il facilite le développement grâce aux extensions C/C++, au débogage, à l'exécution de tâches et à une navigation rapide dans les fichiers du projet.

Git est un système de gestion de versions distribué. Il permet de suivre l'historique du code, gérer les branches, revenir à des versions stables et faciliter le travail en équipe.

GitHub est une plateforme basée sur Git pour héberger les dépôts, collaborer (issues, pull requests), partager le code et centraliser la documentation du projet.

Chapitre 2

Méthodologie et rôle individuel

2.1 Approche méthodologique adoptée au sein de l'équipe

Dans le cadre du projet *Gestion des ressources d'un Data Center*, l'équipe a adopté une démarche de développement itérative, structurée autour de livraisons progressives. Cette approche a permis de valider rapidement une première version fonctionnelle (consultation publique des ressources, authentification), puis d'enrichir progressivement les modules métiers : gestion du catalogue de ressources, workflow de réservation, traitement administratif (approbation/refus) et traçabilité des actions.

Planification et répartition des tâches

Les travaux ont été organisés à partir des exigences du cahier des charges : gestion des ressources (types/états/caractéristiques), système complet de réservation avec statuts, vérification de disponibilité et gestion des conflits (chevauchement temporel), contrôle d'accès par rôles (user/manager/admin), tableau de bord administrateur, ainsi qu'un mécanisme de journalisation. Les tâches ont ensuite été réparties par responsabilités : conception de la base de données, implémentation des modèles Eloquent et des migrations, développement des contrôleurs et des routes, réalisation des vues Blade, et mise en place des services métiers (détection de conflit, journal d'activité) et du middleware d'autorisation.

Développement modulaire et intégration progressive

Afin d'assurer la maintenabilité et de faciliter l'intégration, le projet a été structuré selon l'architecture MVC de Laravel en séparant clairement :

- **Models** (`User`, `Resource`, `Reservation`, `ActivityLog`) pour la représentation des données et des relations ;
- **Controllers** pour la gestion des cas d'usage (consultation, réservation, administration) ;
- **Services** pour isoler la logique métier réutilisable (ex. `ReservationConflictService`, `ActivityLogService`) ;

- **Middleware** pour sécuriser l'accès aux routes d'administration (`IsAdmin`) ;
- **Views Blade** pour l'interface utilisateur (public, utilisateur authentifié, admin).

L'intégration a été réalisée progressivement : d'abord la base de données et les migrations, ensuite la navigation publique (`/resources/browse`), puis l'authentification, ensuite le module de réservation (`/resources/{resource}/reserve` et `/my-reservations`), et enfin l'espace administration (`/admin`) incluant la gestion des ressources et le traitement des demandes.

Choix techniques guidés par la conception

Les choix techniques ont été guidés par la recherche de robustesse et de lisibilité :

- **Eloquent ORM** pour la manipulation des données (relations, filtrage, pagination) ;
- **Migrations** pour versionner le schéma de base de données ;
- **Validation Laravel** (FormRequest et règles de validation) pour sécuriser les entrées utilisateur, notamment les dates de réservation ;
- **Services dédiés** afin d'encapsuler les règles métier critiques : la détection de conflit (overlapping) et la journalisation ;
- **Contrôle d'accès par rôles** via middleware et méthodes du modèle utilisateur, afin de protéger les routes d'administration.

Validation (tests) et démonstration

La validation a été effectuée de manière continue à travers des scénarios de test fonctionnels : création de comptes, consultation des ressources avec filtres, création de réservations, vérification des conflits (réservations qui se chevauchent), annulation d'une demande, traitement administratif (approbation/refus) et contrôle d'accès aux routes `/admin`. En parallèle, la démonstration repose sur des pages clés : liste des ressources, formulaire de réservation, liste des réservations utilisateur, tableau de bord administrateur et journal d'activité.

2.2 Positionnement de notre rôle et responsabilités principales

Dans le cadre du projet *Gestion des ressources d'un Data Center*, notre positionnement a été à la fois **conception** (modélisation des données et règles métier) et **implémentation** (développement Laravel : routes, contrôleurs, services, vues), afin de fournir une application fonctionnelle, sécurisée et maintenable.

Responsabilités principales

Nos responsabilités se sont articulées autour des axes suivants :

- **Modélisation de la base de données** : définition des entités `users`, `resources`, `reservations`, `activity_logs`, ainsi que des relations et contraintes nécessaires au fonctionnement du système.
- **Implémentation du catalogue de ressources** : ajout de la gestion des ressources (création, consultation, filtrage, changement d'état/maintenance) et mise en place des vues d'administration.
- **Système de réservation et workflow** : développement des formulaires et routes de réservation, suivi des statuts (pending, approved, rejected, active, finished, cancelled), et affichage de l'historique des demandes côté utilisateur.
- **Détection des conflits (overlapping)** : intégration d'un service métier `ReservationConflictService` appliquant la règle de chevauchement temporel afin d'éviter la double allocation d'une même ressource.
- **Sécurité et contrôle d'accès** : mise en place de l'authentification, des rôles (user/manager/admin), et protection des routes sensibles via middleware (`auth`, `admin`).
- **Traçabilité** : implémentation du journal d'activité via `ActivityLogService` afin d'enregistrer les actions critiques (création, modification, approbation, rejet, connexion, etc.) et faciliter l'audit.
- **Tableau de bord administrateur** : consolidation des indicateurs de suivi (ressources, réservations, utilisateurs, activités récentes) afin d'offrir une vision synthétique de l'occupation et de l'état du système.

Résultat attendu

À l'issue de nos contributions, le système fournit une plateforme web cohérente où les utilisateurs consultent les ressources, déposent des demandes de réservation et suivent leur état, tandis que l'administration gère le parc matériel, traite les demandes et supervise l'activité globale. L'ensemble respecte les exigences de sécurité (authentification/autorisation), de cohérence métier (prévention des conflits) et de traçabilité (journalisation), tout en restant extensible pour l'ajout futur de notifications internes et de statistiques davantage dynamiques.

Chapitre 3

Conception et Développement

3.1 Conception et Développement du l'Application Web de Réservation des Ressources du Data Center

3.1.1 Objectif et principes de conception

Notre sous-projet vise à mettre en place un **système de réservation, d'allocation et de suivi des ressources** d'un Data Center (serveurs, machines virtuelles, stockage, équipements réseau), afin de garantir une utilisation efficace et transparente du parc informatique. Contrairement à une gestion manuelle ou non centralisée, l'objectif n'est pas uniquement d'enregistrer des demandes, mais d'assurer une **gestion fiable de la disponibilité** en intégrant l'état courant des ressources (*available, reserved, maintenance, down*) et le **cycle de vie complet des réservations** (*pending, approved, rejected, active, finished, cancelled*). Une attention particulière est portée à la **prévention des incohérences** liées aux demandes concurrentes : le système applique une **détection automatique des conflits de réservation** (chevauchement temporel) afin d'éviter la double allocation d'une même ressource. Enfin, la conception prend en compte la sécurité et la gouvernance via un **contrôle d'accès par rôles** (user/manager/admin) et la **traçabilité** des actions importantes (journal d'activité), ce qui renforce l'audit et la responsabilité dans l'exploitation du Data Center.

3.2 Diagrammes UML / Merise

3.2.1 Diagramme de cas d'utilisation (Use Case)

L'application vise la gestion des ressources d'un Data Center et des réservations associées. Les acteurs principaux sont :

- **Invité (Guest)** : accès en lecture seule aux ressources publiques ;
- **Utilisateur (User)** : consultation, réservation, suivi et annulation de ses demandes ;
- **Responsable / Gestionnaire (Manager)** : même périmètre que l'administrateur (accès admin) ;

- **Administrateur (Admin)** : gestion des ressources, traitement des réservations, supervision.

Les cas d'utilisation majeurs, conformément aux routes du projet (`routes/web.php`), sont :

- **Consulter les ressources (public)** : `/resources/browse`, `/resources/view/{resource}` ;
- **S'authentifier** : `/login`, `/logout` et **S'inscrire** : `/register` ;
- **Créer une réservation** : `/resources/{resource}/reserve` (GET/POST) ;
- **Consulter mes réservations** : `/my-reservations` et `/my-reservations/{reservation}` ;
- **Annuler une réservation** : `/my-reservations/{reservation}/cancel` ;
- **Administration** (protégé par `auth` + `admin`) :
  - Tableau de bord : `/admin` ;
  - Gestion des ressources : `/admin/resources` (création, suppression, maintenance, activation/désactivation) ;
  - Gestion des réservations : `/admin/reservations` (approbation/refus) ;
  - Statistiques (vue) : `/admin/statistics` ;
  - Journal d'activité (vue) : `/admin/activity-log`.

3.2.2 Use Case – Gestion des ressources d'un Data Center

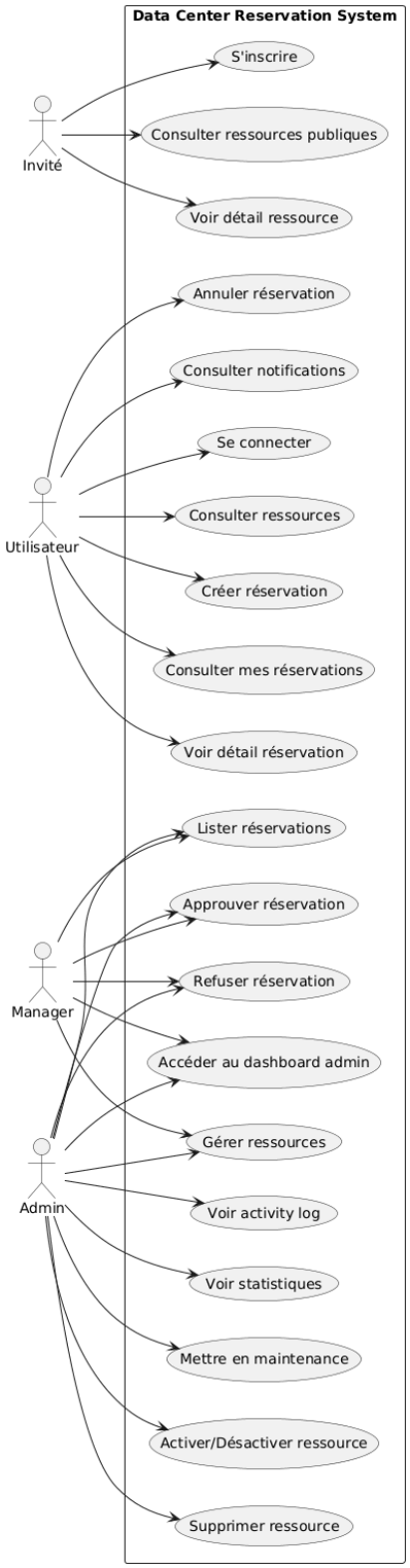

Fig. 3.1: Diagramme de cas d'utilisation du système de réservation des ressources d'un Data Center (rôles : Invité, Utilisateur, Manager, Administrateur)..

3.2.3 Diagramme de classes

Le cœur métier s'articule autour de quatre modèles Eloquent : `User`, `Resource`, `Reservation`, `ActivityLog`. Les relations principales sont :

- **User** (1) — (N) **Reservation** ;
- **Resource** (1) — (N) **Reservation** ;
- **User** (0..1) — (N) **ActivityLog** via `actor_id` nullable ;
- **ActivityLog** cible polymorphe via `target_type` + `target_id`.

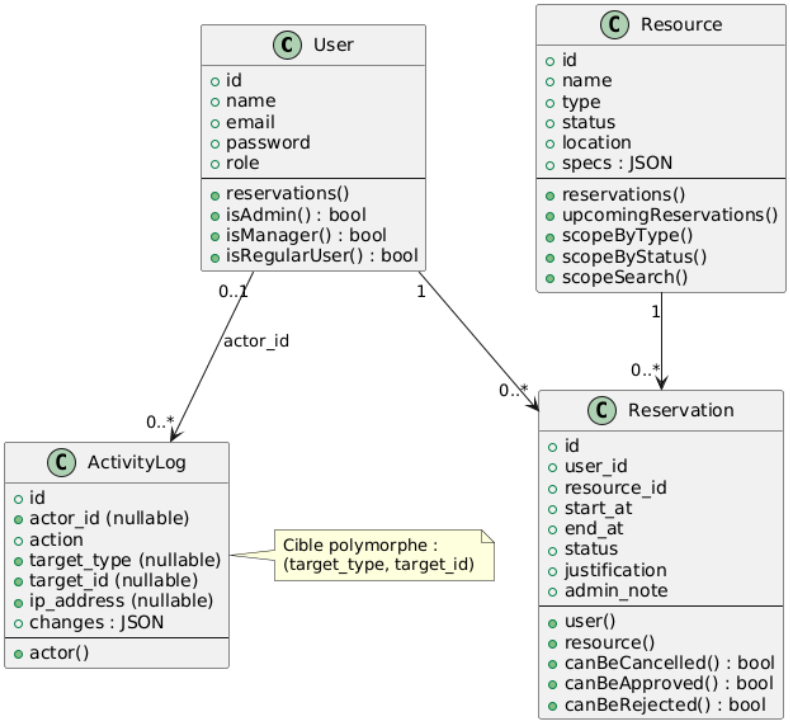

Fig. 3.2: Diagramme de classes du système de réservation des ressources d'un Data Center, présentant les entités principales (*User, Resource, Reservation, ActivityLog*), leurs attributs, méthodes et relations (réservations, journalisation et cible polymorphe)

3.3 Modélisation de la base de données

Description

La base de données a été conçue afin d'assurer une gestion efficace des serveurs, des utilisateurs et des services associés. Elle repose sur une structure relationnelle permettant la cohérence des données et la facilité des opérations **CRUD (Create, Read, Update, Delete)**.

Les principales entités sont :

Utilisateur : représente les administrateurs et les utilisateurs autorisés à accéder au système.

Attributs : id, nom, email, motdepasse, rôle.

Serveur : contient les informations relatives aux serveurs gérés.

Attributs : id, nom, adresseip, système exploitation, statut, datecreation.

Service : représente les services installés ou exécutés sur un serveur.

Attributs : id, nomservice, port, statut, serveur id.

Journal (Logs) : conserve l'historique des actions effectuées sur le système.

Attributs : id, action, date, utilisateurid.

Les relations entre les tables sont définies par des clés étrangères afin de garantir l'intégrité référentielle (par exemple : un serveur peut contenir plusieurs services).

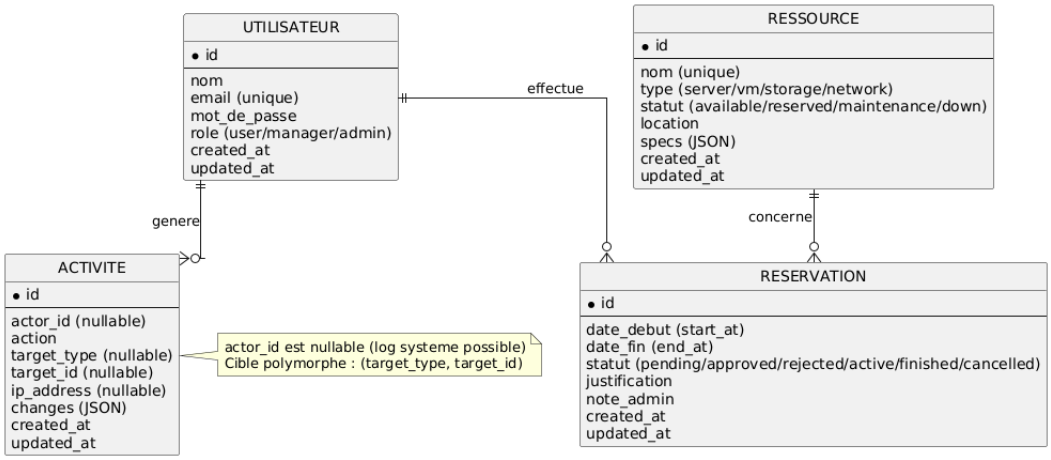

Fig. 3.3: MCD (Merise) du système de réservation des ressources du Data Center

3.3.1 Structure de Projet

Structure des fichiers créés

- **app/**
  - **Models/**
    - **Resource.php** □ Relations + Scopes
    - **Reservation.php** □ Relations + Vérifications
    - **User.php** □ Méthodes helper
    - **ActivityLog.php** □ Traçabilité
  - **Http/**
    - **Controllers/**
      - **ResourceController.php** □ Liste + Détails
      - **ReservationController.php** □ CRUD Réservation
      - **Admin/**
      - **ReservationAdminController.php** □ Approbation/Rejet
    - **Requests/**

        - **StoreReservationRequest.php** □ Validation créa
        - **ApproveReservationRequest.php** □ Auth Admin
        - **RejectReservationRequest.php** □ Validation raison
    - **Middleware/**
        - **IsAdmin.php** □ Vérification rôle
  - **Services/**
    - **ReservationConflictService.php** □ Logique conflit
    - **ActivityLogService.php** □ Logging

- **routes/web.php** □ Toutes les routes

3.3.2 Structure du projet (résumé)

Le projet suit l'architecture MVC de Laravel (Models–Views–Controllers) et est organisé comme suit :

- **app/** : cœur applicatif.
  - **app/Models/** : modèles Eloquent (`User`, `Resource`, `Reservation`, `ActivityLog`).
  - **app/Http/Controllers/** : contrôleurs (Auth, Ressources, Réservations, Admin).
  - **app/Http/Middleware/** : sécurité/autorisation (`IsAdmin`, `AllowGuest`).
  - **app/Services/** : logique métier (`ReservationConflictService`, `ActivityLogService`).
- **routes/** : définition des routes (`web.php`) pour les espaces public, authentifié et admin.
- **resources/** : interface utilisateur.
  - **resources/views/** : vues Blade (auth, resources, reservations, admin, notifications).
  - **resources/css/** et **resources/js/** : assets front.
- **database/** : couche base de données.
  - **database/migrations/** : création/évolution des tables.
  - **database/seeders/** : données de test.
- **public/** : point d'entrée web (`index.php`) et fichiers accessibles publiquement.
- **storage/** : logs, cache, sessions (selon configuration).
- **tests/** : tests unitaires et tests fonctionnels.

3.3.3 Fichiers importants

- **artisan** : outil CLI Laravel (migrate, serve, cache, etc.).
- **composer.json** : dépendances PHP.
- **package.json** / **vite.config.js** : build des assets front.
- **README.md** : procédure d'installation et d'exécution.

Chapitre 4

Cahier des charges

4.1 Contexte et besoin

4.1.1 Objectifs

L'objectif principal de ce projet est de concevoir et développer une application Web permettant la **réservation**, l'**allocation** et le **suivi** des ressources informatiques d'un Data Center (serveurs, machines virtuelles, stockage, équipements réseau). La plateforme doit offrir une gestion centralisée des demandes, en garantissant la disponibilité des ressources sur des périodes définies et en évitant les conflits de réservation (chevauchement temporel).

En parallèle, l'application vise à assurer une **gestion transparente** grâce à un suivi clair du cycle de vie des réservations (*en attente, approuvée, refusée, active, terminée, annulée*) et à fournir une **traçabilité complète** des actions importantes. Cette traçabilité se traduit par la journalisation des opérations critiques (création, modification, approbation, refus, annulation, connexions), afin de renforcer la sécurité, l'audit et la responsabilité dans l'exploitation des ressources du Data Center.

4.1.2 Acteurs et rôles

L'application prend en charge plusieurs profils d'utilisateurs, chacun disposant de droits spécifiques, afin d'assurer une gestion structurée et sécurisée des ressources du Data Center.

Invité L'invité accède à l'application sans authentification. Il peut consulter les ressources disponibles en mode lecture seule (liste et fiche détail), afin de connaître les types de ressources proposées et leurs informations générales. Il peut également déposer une demande d'ouverture de compte via l'inscription, pour obtenir un accès utilisateur et pouvoir effectuer des réservations.

Utilisateur /USER (interne) L'utilisateur authentifié dispose d'un espace personnel lui permettant de consulter les ressources avec des filtres (type, statut, recherche), puis de déposer une demande de réservation en précisant la période souhaitée (date de début et date

de fin) ainsi qu'une justification. Il peut suivre l'avancement de ses demandes grâce aux statuts (*pending, approved, rejected, active, finished, cancelled*), consulter l'historique de ses réservations et annuler une réservation lorsque cela est autorisé. Un espace de notifications permet de centraliser les informations liées aux réponses (approbation/refus) et aux mises à jour importantes.

Administrateur /MANAGER (Admin) L'administrateur (rôle `admin`) possède la supervision complète du système. Il gère l'ensemble du catalogue des ressources, pilote le traitement des réservations, supervise les utilisateurs et contrôle la cohérence globale du système. Il accède au tableau de bord et aux statistiques pour suivre l'état des ressources et l'activité des réservations, et peut consulter le journal d'activité (*Activity Log*) afin d'assurer la traçabilité des actions sensibles (créations, modifications, approbations, refus, annulations, connexions). Il peut également gérer les périodes de maintenance via les changements d'état des ressources (`maintenance`) et effectuer des actions d'activation/ désactivation selon les besoins d'exploitation.

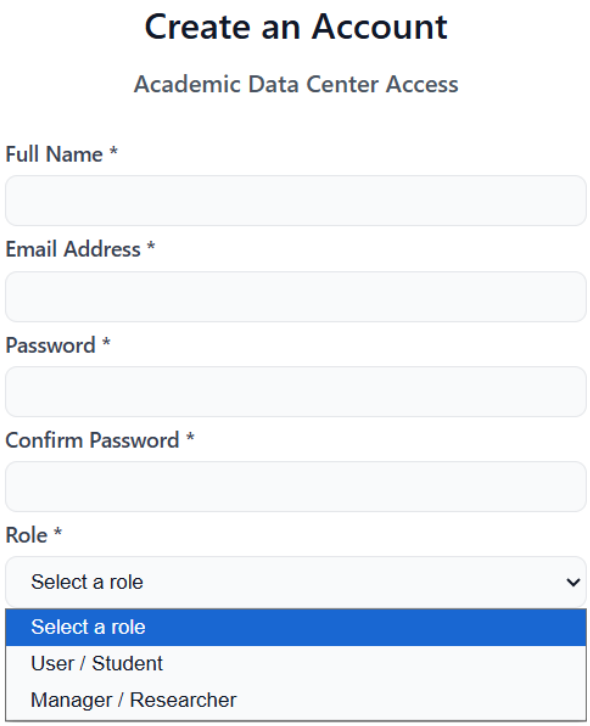

Fig. 4.1: Les roles

4.1.3 Fonctionnalités principales

Le système propose un ensemble de fonctionnalités couvrant l'inventaire des ressources, la réservation, la supervision et l'audit, afin d'assurer une gestion fiable et transparente du Data Center.

Gestion des ressources (catalogue) L'application permet de gérer un catalogue de ressources informatiques structurées par **type** (*server, vm, storage, network*) et par **état** (*available, reserved, maintenance, down*). Chaque ressource dispose d'une **fiche descriptive** comprenant au minimum un nom unique, un emplacement (optionnel) et des caractéristiques techniques (stockées sous forme de JSON : CPU, RAM, stockage, OS, etc.). L'espace d'administration permet d'ajouter, modifier et supprimer des ressources, ainsi que de mettre à jour leur état (par exemple basculer une ressource en *maintenance* lors d'une intervention).

Système de réservation et gestion des conflits Les utilisateurs authentifiés peuvent effectuer une demande de réservation en précisant une **période** (`start_at` / `end_at`) et, si nécessaire, une **justification**. Chaque réservation suit un cycle de vie via des statuts (*pending, approved, rejected, active, finished, cancelled*). Un mécanisme de **détection automatique des conflits** empêche la double allocation d'une même ressource : une demande est rejetée (ou bloquée à l'approbation) lorsqu'un chevauchement temporel (*overlapping*) est détecté avec une réservation existante sur la même ressource.

Tableau de bord et statistiques Un tableau de bord (*Dashboard*) est mis à disposition des rôles administratifs afin de fournir une vision synthétique de l'état du système : nombre d'utilisateurs, nombre de ressources par statut, volume de réservations par état, ainsi qu'un aperçu des demandes en attente. Des pages de statistiques permettent de suivre l'activité globale et d'analyser l'occupation du Data Center selon les indicateurs disponibles.

Logs et traçabilité (audit) L'application intègre un **journal d'activité** (*Activity Log (statique)*) destiné à tracer les actions importantes liées à la sécurité et à l'exploitation : création/modification/ suppression de ressources, création/annulation de réservations, approbation/refus par l'administration, et événements d'authentification. Les logs incluent les informations utiles à l'audit, telles que l'acteur (nullable pour logs système), l'adresse IP, la nature de l'action et, si nécessaire, les changements stockés en JSON. La notion de cible polymorphe (`target_type`, `target_id`) permet de référencer différents objets du système (ressource, réservation, utilisateur).

Notifications(pour un user)/Historique (pour Admin , User ..) Un espace **Notifications** centralise les messages informatifs destinés aux utilisateurs, notamment les retours. sur les demandes (approbation/refus), les changements d'état impactant une réservation, ou d'autres alertes applicatives. Cette messagerie interne vise à améliorer la communication et la réactivité, en évitant la dépendance à des échanges externes et en conservant un historique consultable depuis l'application.

4.1.4 Fonctionnalités implémentées

Le système développé permet de couvrir les fonctionnalités essentielles de la gestion des serveurs :

- Authentification et gestion des utilisateurs (connexion, rôles et permissions).
- Ajout, modification et suppression des serveurs.
- Suivi de l'état des serveurs (actif, inactif, maintenance).
- Gestion des services associés à chaque serveur.
- Consultation de l'historique des actions (logs).
- Interface d'administration simple et intuitive.
- Sécurisation des accès aux fonctionnalités sensibles.

Ces fonctionnalités permettent une administration centralisée et efficace de l'infrastructure serveur.

4.1.5 Choix technologiques

Frontend

Conformément au cahier des charges : CSS/JS Personnalisé : Aucun framework (Bootstrap/Tailwind) n'a été utilisé. Le design est "Vanilla", garantissant un code léger et une maîtrise totale de l'ergonomie. Blade : Le moteur de template de Laravel est utilisé pour générer les vues dynamiquement.

Backend

Nous avons choisi le framework Laravel (PHP) pour son architecture MVC (Modèle-Vue-Contrôleur) qui sépare la logique métier de l'affichage. L'utilisation d'Eloquent ORM sécurise les interactions avec la base de données via des requêtes préparées, tandis que les Middlewares assurent le contrôle d'accès.

Base de données

MySQL a été retenu pour sa fiabilité. Nous utilisons les Migrations pour le versioning du schéma et les Seeders pour le déploiement de données de test.

Chapitre 5

Réalisation

5.1 Implémentation

5.1.1 Architecture du projet

Le projet suit l'arborescence standard de Laravel : app/Models : Définition des entités et relations. app/Http/Controllers : Logique de traitement des requêtes. app/Http/Middleware : Filtres de sécurité (ex : CheckAdmin). routes/web.php : Définition des points d'entrée de l'application

5.1.2 Fonctionnalités

Le workflow principal permet à un utilisateur de filtrer les ressources, de soumettre un formulaire de réservation avec justification, et au responsable de changer le statut en un clic, déclenchant une notification instantanée

5.1.3 Règles de gestion importantes

Conflits de réservation (overlapping)

La règle principale de cohérence métier est la prévention des chevauchements temporels sur une même ressource. Deux réservations sont considérées en conflit si leurs intervalles se recoupent. Cette règle est appliquée lors de la création d'une réservation et/ou lors de l'approbation, afin d'empêcher toute double allocation.

Gestion des statuts

La réservation suit un cycle de vie contrôlé par statuts. Les transitions typiques sont : `pending` → `approved` ou `rejected`, puis éventuellement `active` et `finished`. Une annulation explicite entraîne `cancelled`. Cette logique garantit un suivi clair et évite les incohérences (ex. approuver une réservation déjà annulée).

Permissions (RBAC)

Le contrôle d'accès repose sur un modèle RBAC (*Role-Based Access Control*). Les actions sont autorisées selon le rôle : l'utilisateur standard gère ses réservations, tandis que les rôles administratifs traitent les demandes et gèrent les ressources. Les routes d'administration sont protégées par middleware afin d'empêcher l'accès non autorisé.

5.2 Sécurité et contrôle d'accès

5.2.1 Middleware d'authentification

Les fonctionnalités sensibles sont accessibles uniquement après authentification. Les routes réservées aux utilisateurs connectés exigent une session valide. Cette protection empêche l'accès direct aux formulaires de réservation, à l'historique personnel et aux pages internes sans connexion préalable.

5.2.2 Rôles et permissions

Les rôles (`user`, `manager`, `admin`) structurent les droits dans l'application. Le rôle `user` se limite aux opérations sur ses propres réservations. Les rôles `manager` et `admin` disposent d'autorisations élevées : gestion des ressources, accès aux réservations globales, approbation/refus et supervision via le dashboard.

5.2.3 Protection des routes d'administration

Les routes commençant par `/admin` sont protégées par un middleware dédié (ex. `IsAdmin`) qui vérifie que l'utilisateur possède un rôle autorisé (`admin` ou `manager`). En cas d'accès non autorisé, l'application renvoie une erreur (ex. 403) ou redirige selon la stratégie choisie.

5.2.4 Validation des formulaires

Les données envoyées via les formulaires (connexion, inscription, réservation) sont validées côté serveur afin d'éviter les entrées invalides (dates incohérentes, champs manquants, formats incorrects). La validation renforce la robustesse et réduit les risques d'injection ou de comportements inattendus.

5.2.5 Gestion des sessions (si stockées en base)

La persistance de session peut être assurée via la table `sessions`. Cette approche facilite le suivi des connexions et permet une meilleure maîtrise de la sécurité (expiration, invalidation). Elle s'intègre au mécanisme standard de Laravel pour conserver l'état utilisateur entre les requêtes.

5.3 Tests et scénarios

5.3.1 Scénarios de test

Les scénarios de test suivants ont été appliqués pour valider le comportement global du système et la cohérence des règles métier :

- **Réservation valide** : un utilisateur sélectionne une ressource disponible, choisit une période sans chevauchement et soumet la demande. Résultat attendu : création d'une réservation avec statut `pending`, puis approbation possible par l'administration.
- **Réservation en conflit** : un utilisateur tente de réserver la même ressource sur une période qui chevauche une réservation existante. Résultat attendu : détection du conflit (*overlapping*) et blocage de la création ou de l'approbation selon le workflow.
- **Annulation** : l'utilisateur annule une réservation autorisée (non terminée et non déjà annulée). Résultat attendu : mise à jour du statut vers `cancelled` et journalisation de l'action.
- **Approve / Reject** : un administrateur/manager traite une réservation en attente. Résultat attendu : passage en `approved` ou `rejected` (avec note si prévue), sans incohérence de période ni double allocation.
- **Ressource en maintenance** : l'administration change le statut d'une ressource en `maintenance`. Résultat attendu : la ressource devient indisponible à la réservation (ou la réservation est bloquée selon la règle implémentée), tout en conservant l'historique et la traçabilité des changements.

Chapitre 6

Les Captures d'écran de L'Application

6.1 DataCenter Reservation

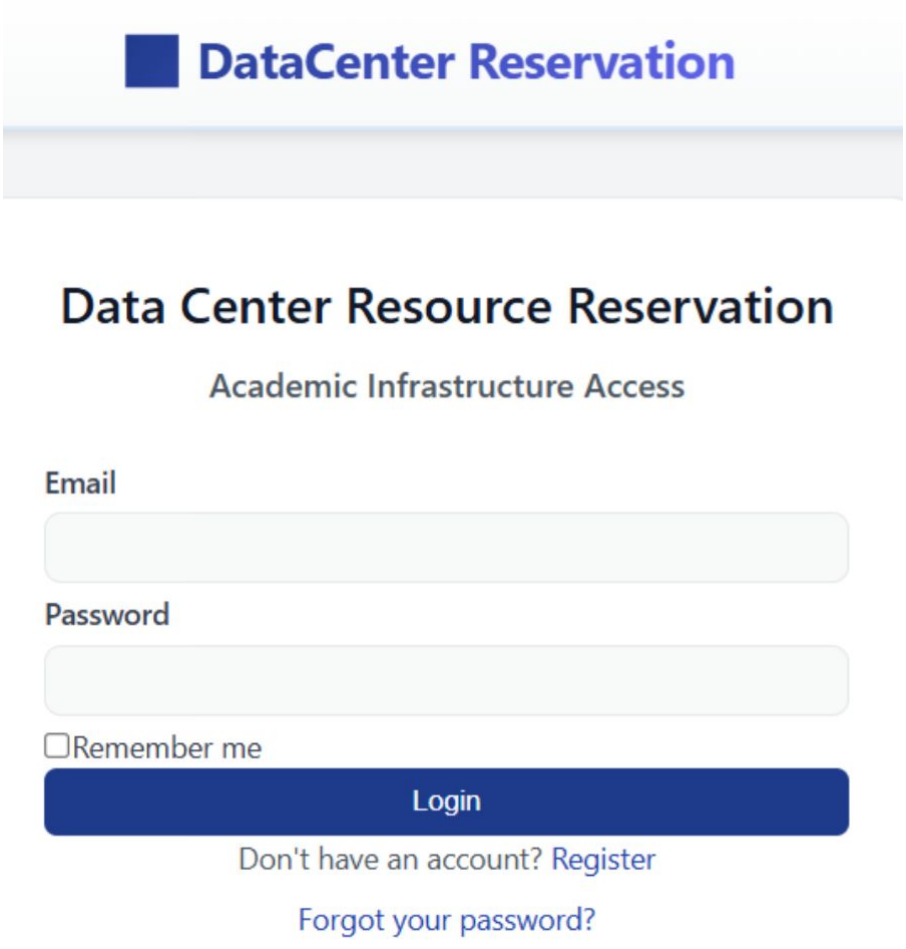

Fig. 6.1: Interface d'authentification (login)

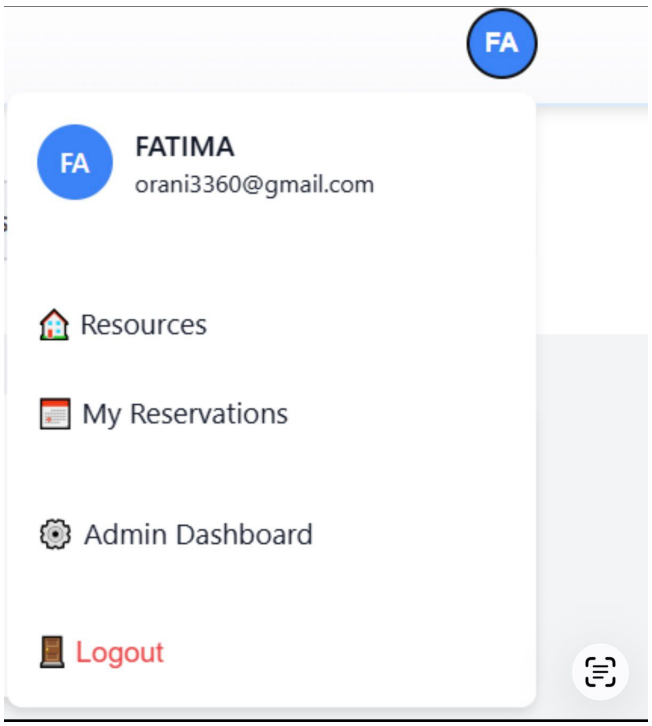

Fig. 6.3: Interface d'un compte cree

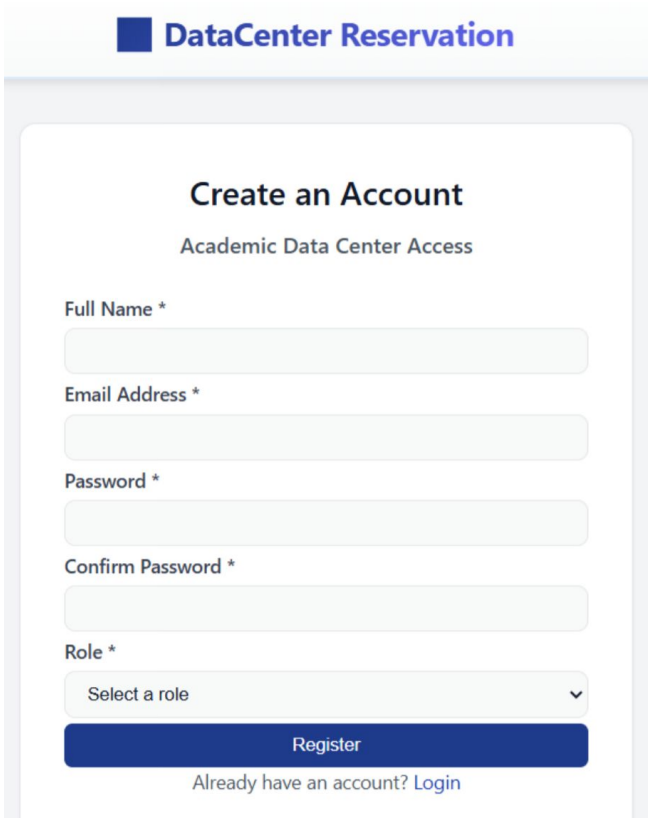

Fig. 6.2: Interface pour Cree un Compte

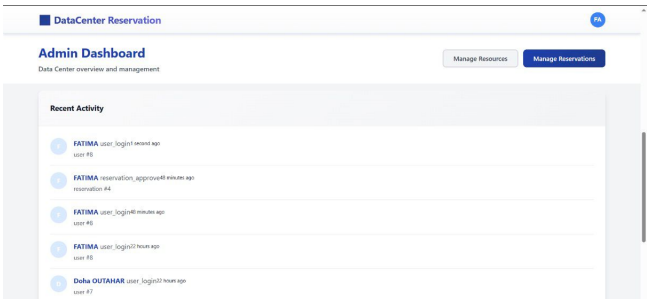

Fig. 6.4: Interface d'historique des actions realiser dans l'application .

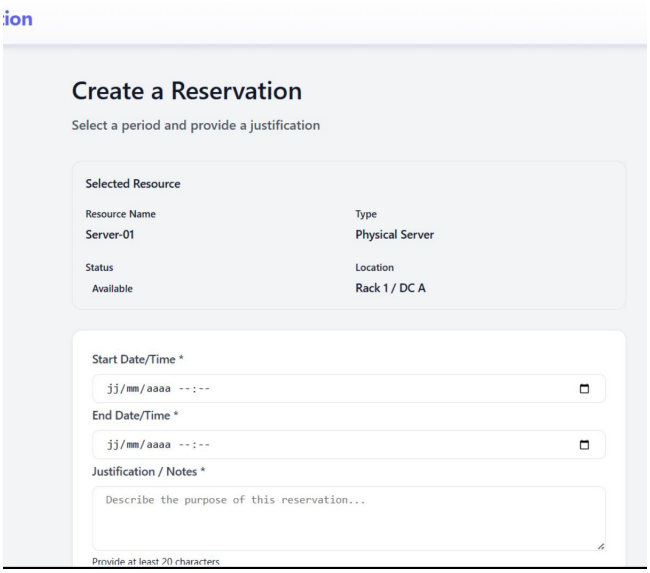

Fig. 6.5: Interface de Creation d'une reservation

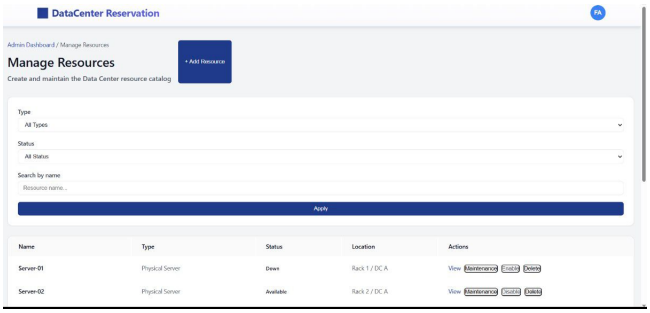

Fig. 6.6: Interface de Management des ressources

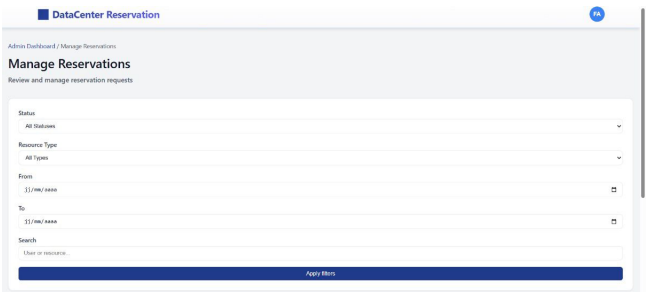

Fig. 6.7: Interface de Management des reservations

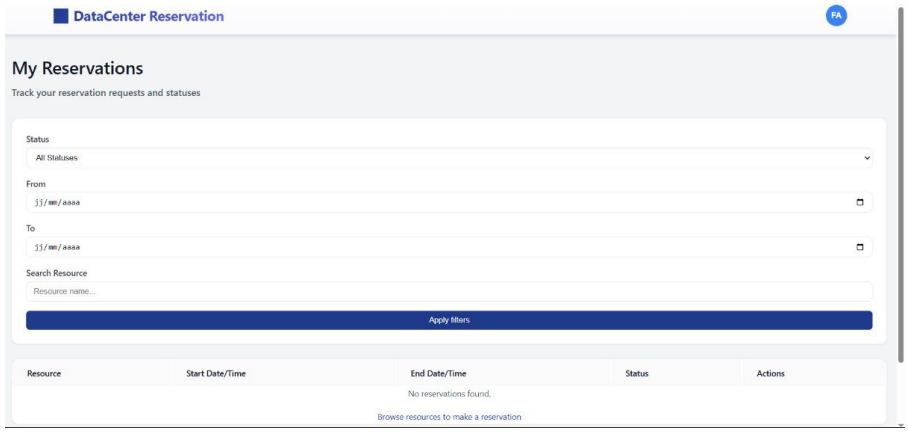

Fig. 6.8: Interface de suivis d'une reservation

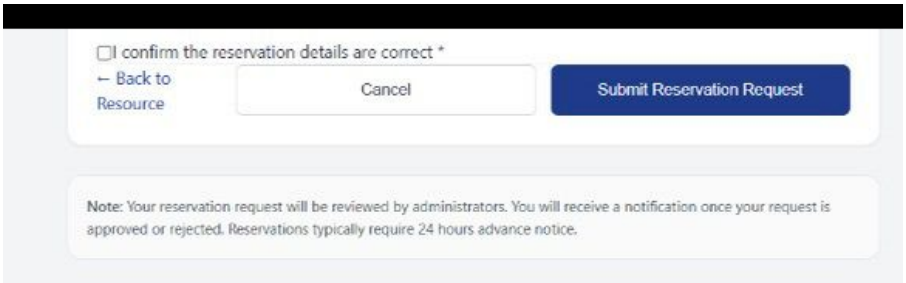

Fig. 6.9: Interface de Confirmation des details d'une Reservation dans le Data Center

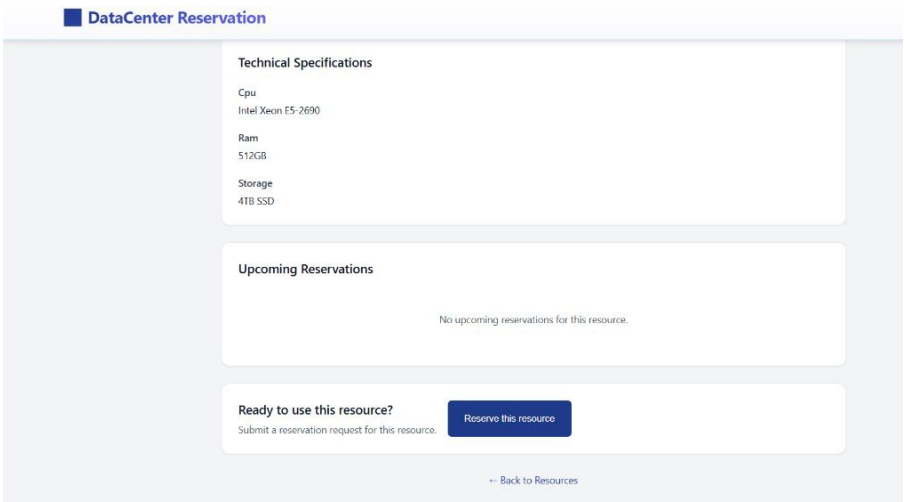

Fig. 6.10: Interface des details Techniques d'un resource

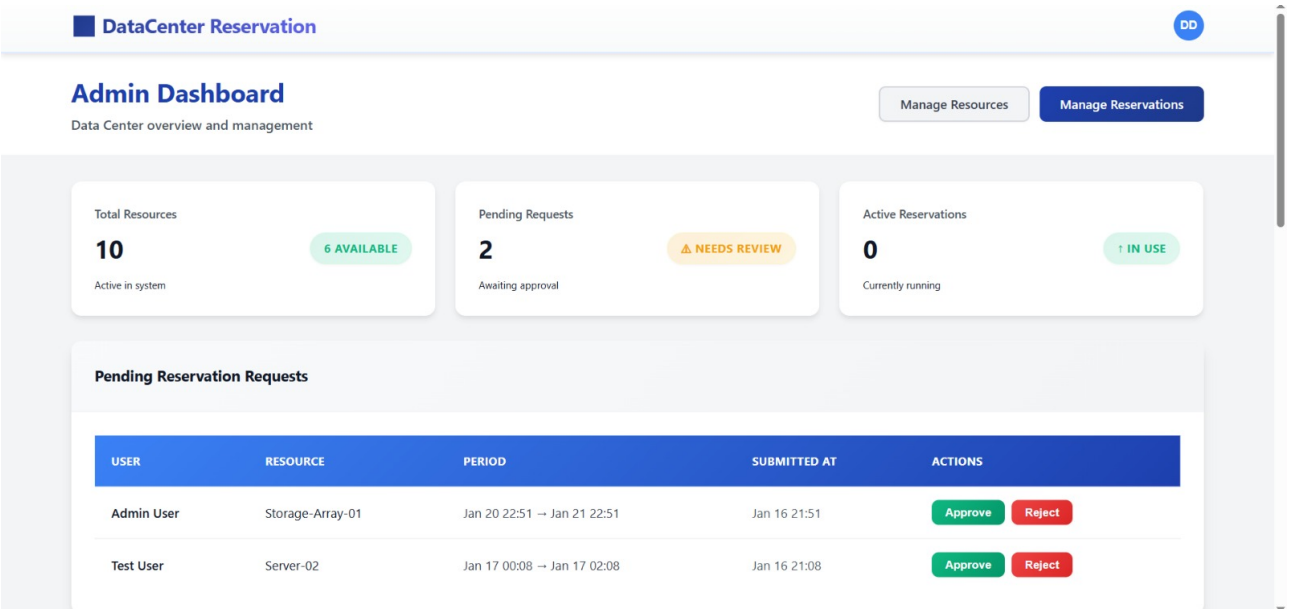

Fig. 6.11: Interface d'administration pour la gestion des reservations

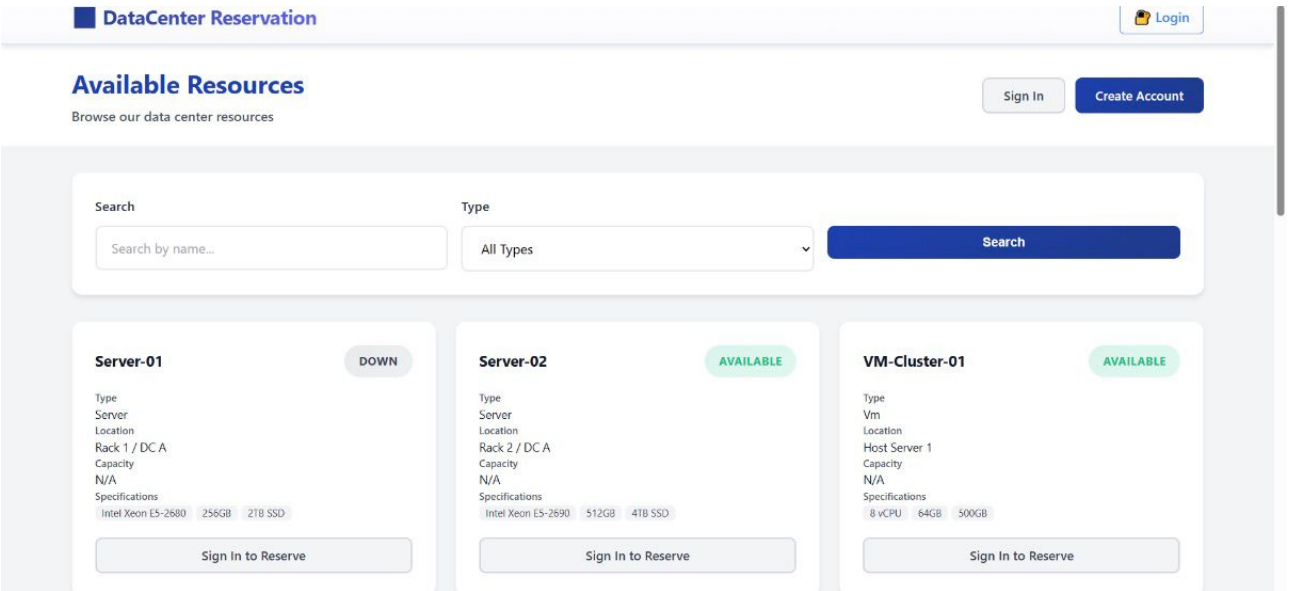

Fig. 6.12: Interface apres log out (switch automatique au mode invite)

Chapitre 7

Conclusion générale

7.1 Conclusion

Ce projet a permis de concevoir et de réaliser une application Web dédiée à la gestion des ressources d'un Data Center, en couvrant les besoins essentiels de **réservation**, d'**allocation** et de **suivi** des ressources (serveurs, VM, stockage, équipements réseau). Grâce à une modélisation relationnelle claire et à l'utilisation de Laravel avec MySQL, la solution mise en place assure un fonctionnement structuré autour de workflows de réservation (création, validation, refus, annulation) et d'un suivi par statuts.

L'application apporte également des garanties importantes en matière de **sécurité** et de **contrôle d'accès** par rôles (user/manager/admin), tout en renforçant la **traçabilité** via la journalisation des actions (*Activity Log*). L'intégration d'un tableau de bord et d'indicateurs de suivi améliore la visibilité sur l'état du parc matériel et l'activité des réservations, tandis que l'espace de notifications contribue à fluidifier la communication interne autour des demandes.

En perspective, plusieurs améliorations peuvent être envisagées, notamment l'enrichissement des notifications (automatisation plus poussée), la gestion détaillée des périodes de maintenance planifiée, ainsi que l'ajout de statistiques graphiques plus avancées (taux d'occupation par type et par période). Globalement, le système répond aux objectifs du cahier des charges en proposant une gestion centralisée, transparente et fiable des ressources informatiques du Data Center.